

Lab Log Book (Week 11)

Name : Jagadeesh

In this step, I reshaped the training data so it could be used by an LSTM model, which requires a 3-dimensional input format. I built a simple LSTM network with 32 units, a dropout layer, and a sigmoid output layer for binary classification. The model was trained for 10 epochs using the Adam optimizer and binary cross-entropy loss.

During training, both accuracy and loss steadily improved. The validation accuracy reached around 0.86, while the validation loss continued to decrease, showing that the model was learning effectively without major overfitting. These results confirm that the LSTM model successfully captured patterns in the dataset and was ready for testing and evaluation.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

X_train_lstm = X_train_imb.reshape((X_train_imb.shape[0], X_train_imb.shape[1], 1))
X_test_lstm = X_test_imb.reshape((X_test_imb.shape[0], X_test_imb.shape[1], 1))

model_lstm = Sequential()
model_lstm.add(LSTM(32, return_sequences=False, input_shape=(X_train_lstm.shape[1], 1)))
model_lstm.add(Dropout(0.2))
model_lstm.add(Dense(1, activation='sigmoid'))

model_lstm.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

history_lstm = model_lstm.fit(
    X_train_lstm, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

649/649	2s	1ms/step	- accuracy: 0.7850	- loss: 0.5136	- val_accuracy: 0.8347	- val_loss: 0.4116
Epoch 2/10						
649/649	1s	1ms/step	- accuracy: 0.8328	- loss: 0.4042	- val_accuracy: 0.8401	- val_loss: 0.3911
Epoch 3/10						
649/649	1s	1ms/step	- accuracy: 0.8411	- loss: 0.3864	- val_accuracy: 0.8457	- val_loss: 0.3773
Epoch 4/10						
649/649	1s	1ms/step	- accuracy: 0.8407	- loss: 0.3874	- val_accuracy: 0.8473	- val_loss: 0.3721
Epoch 5/10						
649/649	1s	1ms/step	- accuracy: 0.8443	- loss: 0.3792	- val_accuracy: 0.8521	- val_loss: 0.3657
Epoch 6/10						
649/649	1s	1ms/step	- accuracy: 0.8512	- loss: 0.3728	- val_accuracy: 0.8498	- val_loss: 0.3680
Epoch 7/10						
649/649	1s	1ms/step	- accuracy: 0.8487	- loss: 0.3729	- val_accuracy: 0.8587	- val_loss: 0.3577
Epoch 8/10						
649/649	1s	1ms/step	- accuracy: 0.8550	- loss: 0.3648	- val_accuracy: 0.8614	- val_loss: 0.3523
Epoch 9/10						
649/649	1s	1ms/step	- accuracy: 0.8598	- loss: 0.3608	- val_accuracy: 0.8600	- val_loss: 0.3525
Epoch 10/10						
649/649	1s	1ms/step	- accuracy: 0.8598	- loss: 0.3546	- val_accuracy: 0.8642	- val_loss: 0.3477

```

: y_pred_lstm_prob = model_lstm.predict(X_test_lstm)
y_pred_lstm = (y_pred_lstm_prob >= 0.5).astype(int)

print("LSTM Classification Report:")
print(classification_report(y_test, y_pred_lstm))

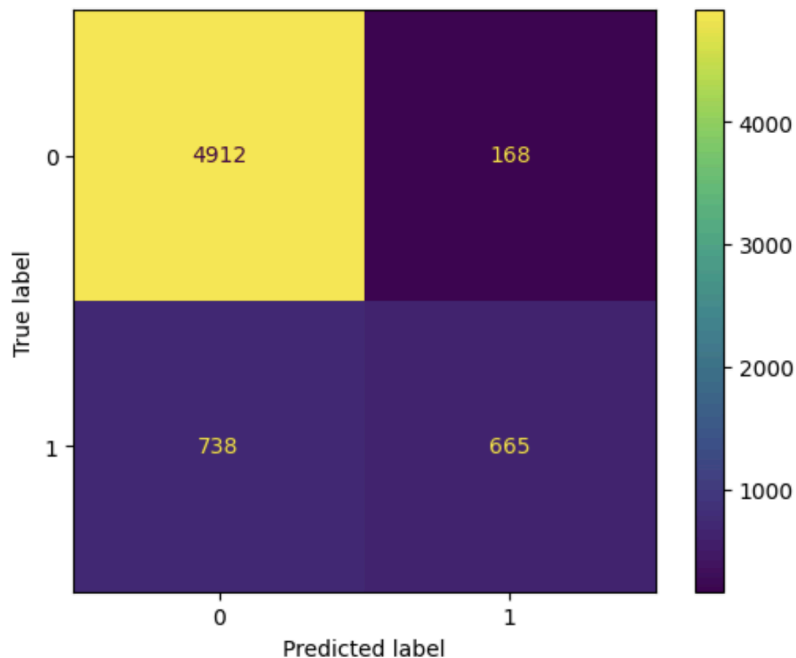
cm_lstm = confusion_matrix(y_test, y_pred_lstm)
disp_lstm = ConfusionMatrixDisplay(confusion_matrix=cm_lstm, display_labels=[0, 1])
disp_lstm.plot()
plt.show()

```

203/203 ————— 0s 873us/step

LSTM Classification Report:

	precision	recall	f1-score	support
0	0.87	0.97	0.92	5080
1	0.80	0.47	0.59	1403
accuracy			0.86	6483
macro avg	0.83	0.72	0.76	6483
weighted avg	0.85	0.86	0.85	6483



The confusion matrix shows how well the LSTM model predicted each class. For the non-default class (0), the model performed strongly- it correctly predicted 4912 cases and only misclassified 168 of them as defaults. This means the model is very reliable at identifying customers who are not likely to default.

For the default class (1), the model was less accurate. It correctly identified 665 default cases but misclassified 738 of them as non-default. This happens because the dataset is imbalanced and contains far fewer default cases.

Overall, the confusion matrix shows that the model is very good at predicting non-default borrowers.

```
[96]: # Compute metrics for LSTM
acc_lstm = np.round(accuracy_score(y_test, y_pred_lstm) * 100, 2)
pre_lstm = np.round(precision_score(y_test, y_pred_lstm, average='weighted') * 100, 2)
rec_lstm = np.round(recall_score(y_test, y_pred_lstm, average='weighted') * 100, 2)
f1_lstm = np.round(f1_score(y_test, y_pred_lstm, average='weighted') * 100, 2)
auc_lstm = np.round(roc_auc_score(y_test, y_pred_lstm) * 100, 2)

# Add LSTM row
model_data_imb.loc[len(model_data_imb.index)] = [
    'LSTM', acc_lstm, pre_lstm, rec_lstm, f1_lstm, auc_lstm
]

model_data_imb
```

[96]:	Imbalance AI Models	Accuracy	Precision	Recall	F1-Score	AUC
0	Random Forest	91.38	91.58	91.38	90.75	81.60
1	Logistic Regression	83.93	82.69	83.93	82.32	69.00
2	MLP	88.99	88.84	88.99	88.10	77.57
3	LSTM	86.02	85.40	86.02	84.61	72.05

The LSTM achieved an accuracy of 86.02% which is slightly lower than Random Forest and MLP but higher than Logistic Regression.

Its precision (85.40%) and recall (86.02%) are also strong and show that the model can correctly identify most non-default and some default cases.

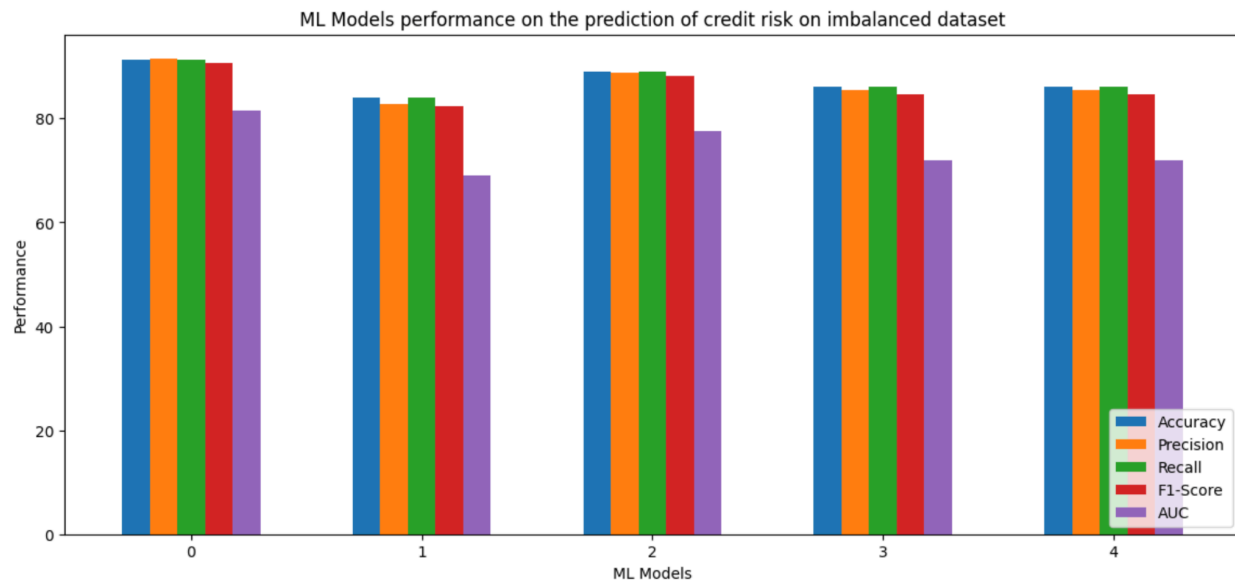
The F1-score (84.61%) reflects good overall balance between precision and recall.

The AUC value of 72.05% shows that the LSTM is reasonably good at distinguishing between default and non-default borrowers, though not as strong as Random Forest or MLP.

```

: model_data_imb.plot(kind='bar', figsize=(14,6), width=0.6)
plt.title('ML Models performance on the prediction of credit risk on imbalanced dataset')
plt.ylabel('Performance')
plt.xlabel('ML Models')
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.show()

```



From the chart, Random Forest shows the strongest overall performance, with high scores across all metrics. The MLP model also performs well and stays close to Random Forest. Logistic Regression has noticeably lower AUC and recall compared to the others. The LSTM model performs competitively, especially in accuracy and recall, but its AUC is lower than Random Forest and MLP.